

Lecture 1

Introduction to Backend Development

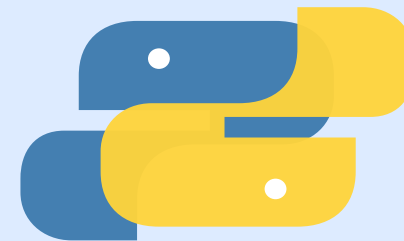
Backend Python + Django Course
Second Year – Faculty of Computers & AI

This lecture builds the foundation before coding:
web architecture, backend roles, Python setup, VS
Code setup, terminal basics, and the first program.

Dr. Mohamed Nour Eldien

...

•
•
•



Lecture roadmap

1. What backend development means
2. How a website works from browser to server
3. Frontend vs backend vs database
4. Static vs dynamic web pages
5. Why Python is a good backend language
6. What a framework is and why Django matters
7. Installing Python and VS Code
8. Writing and running the first Python program
9. Common beginner mistakes and quick practice

Teaching tip: tell students that everything in Django will make more sense after they understand the request-response cycle.

Learning outcomes

By the end of this lecture, a student should be able to:

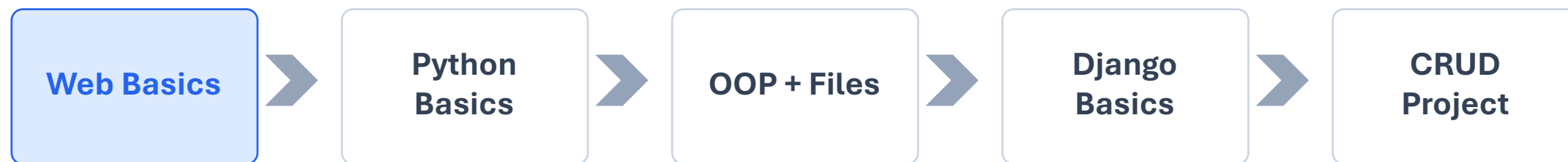
Conceptual outcomes

- Define backend development in simple words.
- Draw the relationship between browser, server, and database.
- Explain the difference between static and dynamic websites.
- Explain why Python is popular in backend development.

Practical outcomes

- Install Python successfully on a computer.
- Install Visual Studio Code and useful extensions.
- Open a terminal and navigate to a folder.
- Create, save, and run the first Python file.

Where this lecture fits in the course



Lecture 1 is the foundation. Students who understand this lecture will understand later Django lectures much faster.

What is backend development?

- Backend development is the part of a software system that runs on the server side.
- It receives requests, processes logic, talks to databases, enforces security, and returns responses.
- The backend is not usually visible to end users, but it controls the application behavior.

-
-
-

**Think of the backend as the
“brain” of the application.**

Use examples students know: university portal, e-commerce checkout, social media login.

Main responsibilities of a backend

Data and logic

- Store and retrieve data from a database.
- Validate user input before saving it.
- Apply business rules, such as login checks or grade calculation.
- Generate dynamic content based on user data.

Security and integration

- Authenticate users and manage permissions.
- Handle errors and send appropriate messages.
- Connect the application to email, payment, or other APIs.
- Serve data to web pages or mobile apps.

Daily-life examples of backend work

1. Logging in to a website

2. Searching for a product

3. Submitting a registration form

4. Saving a new post or comment

5. Checking if a username already exists

6. Generating a personalized dashboard

Frontend vs Backend

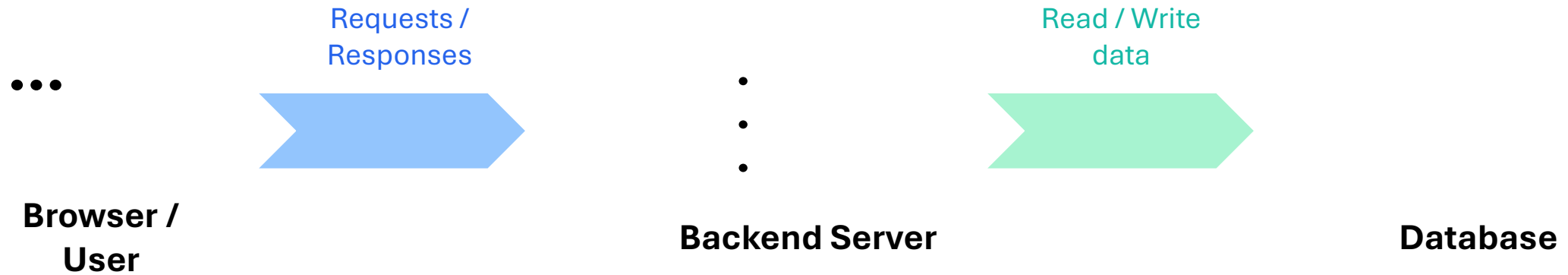
Frontend (client side)

- What the user sees and interacts with.
- Built using HTML, CSS, and JavaScript.
- Runs mainly inside the browser.
- Focuses on layout, colors, buttons, forms, and user experience.

Backend (server side)

- Processes requests from users.
- Built using languages like Python, JavaScript, Java, PHP, or C#.
- Runs mainly on the server.
- Focuses on logic, security, and data handling.

Where does the database fit?



Explain: the database does not talk directly to the user; the backend is the middle layer.

The 3-tier view of a web application

Presentation Layer

HTML, CSS, JS
Browser interface

Application Layer

Python, Django
Rules and processing

Data Layer

SQLite, MySQL,
PostgreSQL
Persistent storage

Static website vs Dynamic website

Static website

- Shows the same content to all users.
- Pages are usually plain HTML files.
- Good for simple informational websites.
- Changes require editing files manually.

Dynamic website

- Content can change based on the user or the database.
- Generated by backend code.
- Good for portals, stores, dashboards, and social apps.
- Can process forms, logins, and user-specific data.

Example: static page behavior

Static HTML file

```
1 <html>
2   <body>
3     <h1>Welcome to the department
  website</h1>
4     <p>Office hours: 9:00 AM - 2:00 PM</p>
5   </body>
6 </html>
```

- Everyone sees the same page content.
- There is no logic deciding what to show.
- No database is required for this simple page.

Example: dynamic page behavior

Backend creates dynamic content

```
1 def dashboard(request):  
2     student_name = 'Mona'  
3     cgpa = 3.8  
4     return render(request, 'dashboard.html',  
5         {  
6             'student_name': student_name,  
7             'cgpa': cgpa  
8         })
```

- The page can display different content for each student.
- The backend can fetch data from a database first.
- Dynamic websites are the norm in modern applications.

How does a website work?

1. The user enters a URL or clicks a link.
2. The browser sends an HTTP request to the server.
3. The backend receives the request and decides what to do.
4. The backend may read from or write to the database.
5. The server returns an HTTP response.
6. The browser displays the page to the user.

The request-response cycle



What is inside a request?

Common request parts

- HTTP method such as GET or POST.
- URL path such as /students or /login.
- Headers that carry metadata.
- Optional body data in forms or JSON.

Why requests matter

- The method tells the backend the intended action.
- The URL tells the backend which resource is requested.
- The body contains user-submitted information.
- Django reads all of this through the request object.

What is inside a response?

Common response parts

- Status code such as 200 or 404.
- Headers that describe the response.
- Response body such as HTML, JSON, or plain text.
- Cookies or session data if needed.

Why responses matter

- The browser decides how to display the content.
- The status code tells the browser whether the request succeeded.
- APIs often return JSON instead of HTML.
- In Django, views usually return an `HttpResponse` or render a template.

HTTP methods: GET and POST are the first two you need

GET

Used to request and read information.

Examples: open a page, view student list,
search with URL parameters.

GET requests should not change important
data.

POST

Used to send data to the server.

Examples: submit a login form, create a
new student, upload data.

POST is commonly used when data
changes are expected.

Three status codes every beginner should know

200 OK

The request
succeeded.

404 Not Found

The page or resource
does not exist.

500 Server Error

Something failed
inside the backend
code.

Anatomy of a URL

`https://example.com/students/list/?level=2`

Protocol

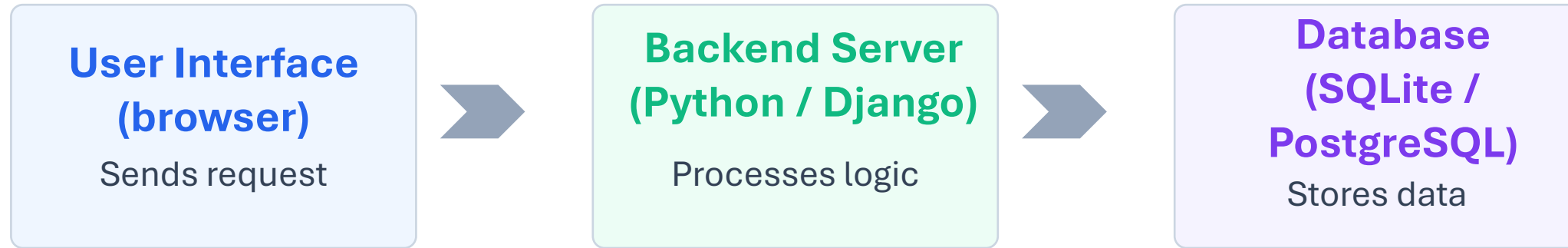
Domain

Path

Query string

Later in Django, URLs are mapped to views using URL patterns.

Browser, server, and database: one complete picture



Why backend development matters in real systems

- Without backend code, a website cannot save student accounts, login data, exam records, or dynamic content.
- The backend protects data and makes sure that different users see the right information.
- Large systems such as banking apps, hospital systems, and university portals depend heavily on backend logic.
- For developers, backend skills open jobs in web development, APIs, data systems, DevOps-adjacent work, and more.

Why is Python a strong choice for backend development?

Reasons students like Python

- Readable syntax and simple structure.
- Large community and many tutorials.
- Strong libraries for web, data, AI, and automation.
- Good for beginners and still used in real companies.

Reasons teachers like Python

- Students focus on logic instead of complex syntax.
- Easy transition from simple scripts to full frameworks.
- Fast prototyping in labs and assignments.
- Python connects well with databases, APIs, and machine learning.

Python readability compared with intent

Simple readable Python

```
1 name = input('Enter your name: ')
2 if name:
3     print('Welcome', name)
4 else:
5     print('Name is required')
```

- The code is close to natural language.
- Indentation is part of the language syntax.
- This helps beginners write structured code early.

What is a framework?

- A framework is a collection of ready-made tools, rules, and structure that helps developers build applications faster.
- Instead of writing everything from scratch, developers use the framework to organize code and solve common problems.
- A framework usually handles routing, templates, database access, security, and project structure.

**Framework = a
professional starting
structure**

What is Django?



Django is a high-level Python web framework that helps developers build secure and maintainable websites quickly.

- It follows the “batteries included” idea: many useful tools are available from the start.
- It supports URLs, views, templates, models, forms, admin, and authentication.

Keep this lecture focused on introduction; students will install Django in a later lecture.

Why many teachers and companies choose Django

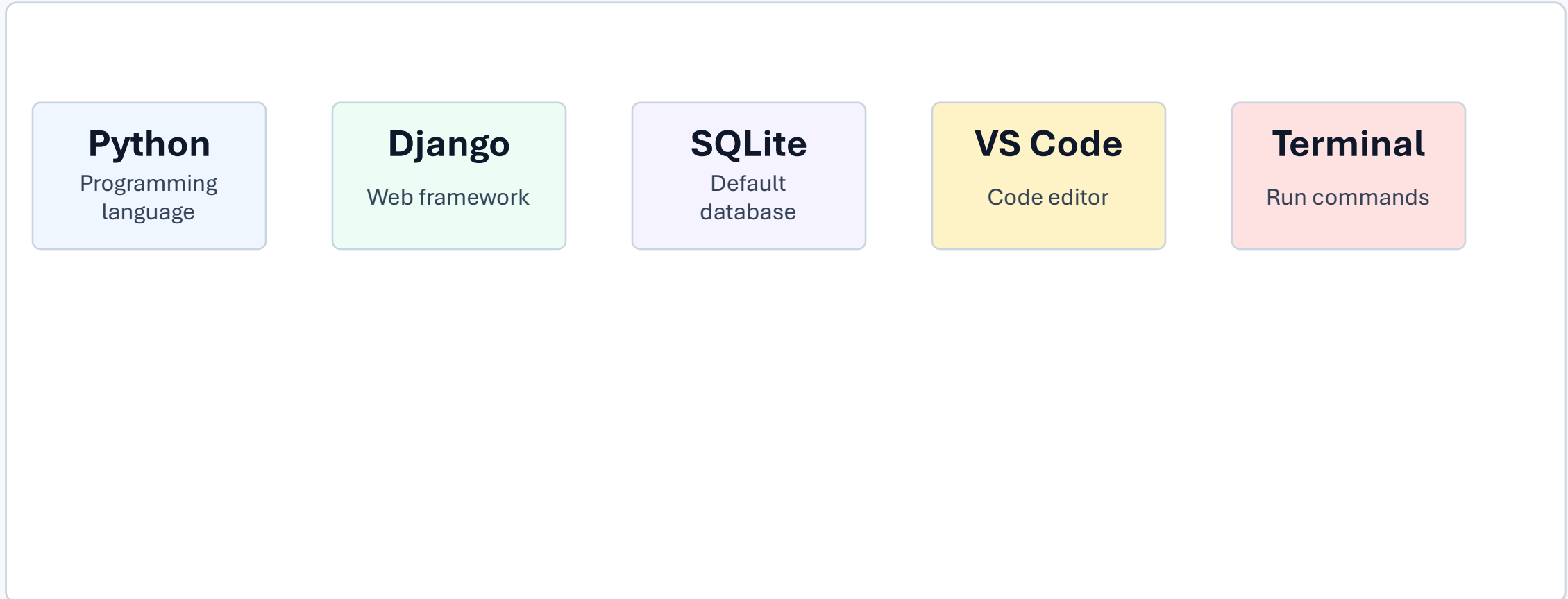
Educational value

- Clear project structure.
- Encourages separation of concerns.
- Comes with an admin panel.
- Good documentation and examples.

Professional value

- Built-in security helpers.
- ORM for database work.
- Scales from small to large projects.
- Large ecosystem and community support.

Typical backend stack for this course



From concept to practice

Now we move from theory to tool setup

In the rest of this lecture, students will prepare the programming environment and run the first Python file.

- Step 1: Install Python
- Step 2: Install VS Code
- Step 3: Open a project folder
- Step 4: Create a .py file
- Step 5: Run the file in the terminal

Before installing anything

What students should have

- A Windows, Linux, or macOS computer.
- Internet connection for downloading installers.
- Permission to install software.
- A folder where course files will be saved.

What teachers should remind them

- Use a recent Python 3 version.
- Do not install random extensions before basics.
- Keep file names simple: avoid spaces at first.
- Check whether Python was added to PATH.

Installing Python: overview

1. Go to the official Python website.
2. Download the latest stable Python 3 installer.
3. Run the installer.
4. Important: enable “Add Python to PATH”.
5. Click “Install Now”.
6. After installation, verify using the terminal.

Mention that Python installers differ slightly by operating system, but the logic is similar.

Python installation screen: what to look for

Python Installer

☒ Add Python to PATH

Install Now

- The check box is critical for beginners.
- Without PATH, the terminal may not recognize python commands.
- This slide is an illustration, not the exact installer screenshot.

How to verify that Python is installed correctly

Terminal command

```
1 python --version  
2 # or on some systems  
3 python3 --version
```

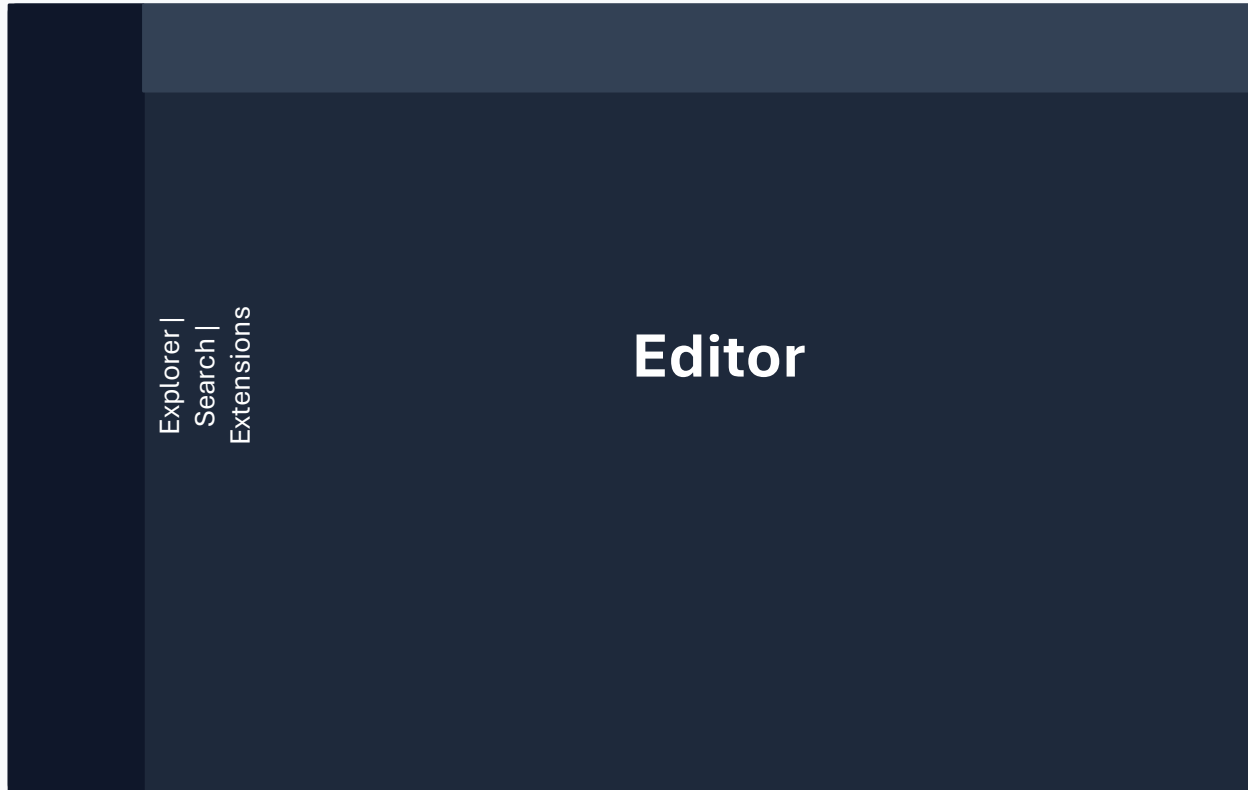
- Open Command Prompt, PowerShell, or Terminal.
- Type the version command.
- If Python is installed correctly, the terminal prints a Python 3.x version.
- If the command is not recognized, PATH may not be configured properly.

Installing Visual Studio Code (VS Code)

1. Go to the official VS Code website.
2. Download the installer for your operating system.
3. Run the installer and follow the setup wizard.
4. Open VS Code after installation.
5. Later, install the Python extension inside VS Code.

VS Code is a code editor, not a programming language and not a framework.

VS Code interface: main areas



- Left sidebar: Explorer and Extensions.
- Center area: code editor.
- Top tabs: opened files.
- Bottom panel: terminal, output, and problems.

Useful extension: Python for VS Code

Extensions
Search:
Python

Install

- Open the Extensions icon in the left sidebar.
- Search for “Python”.
- Install the extension published by Microsoft.
- This extension provides syntax highlighting, linting, and easier execution.

Create a course folder before coding

Example folder name: backend_course

- Inside the folder, students can save all lecture files.
- Organized folders make later Django projects easier to manage.
- A clean folder structure reduces confusion in labs and homework.

Open a folder in VS Code

1. File → Open Folder
2. Choose the folder created for the course
3. Click Select Folder / Open
4. The folder appears in the Explorer panel on the left

Create the first Python file

New file name

```
1 hello.py
```

- In the Explorer panel, click New File.
- Type the file name hello.py.
- The .py extension tells the editor this is a Python file.

Write the first Python program

hello.py

```
1 print('Hello, Backend World!')
```

- The print function displays text in the terminal.
- Strings are written inside quotes.
- Save the file before running it.

Save before run

Keyboard shortcut: Ctrl + S

Unsaved files may lead to confusion because the terminal runs the old version of the code, not the changes the student just typed.

What is the terminal?

Simple definition

- The terminal is a text-based interface for running commands.
- It allows developers to run Python files, install packages, and create projects.
- The terminal is a normal part of programming work.

In this course

- We will use it to run Python files.
- Later, we will use it to create Django projects.
- Students should not fear it; they only need a few commands at first.

How to open the terminal inside VS Code

Method 1: Terminal → New Terminal

Method 2: Shortcut Ctrl + `

The terminal usually opens at the bottom of the VS Code window.

Show this live in class. Students usually understand faster by seeing the terminal panel appear.

Run the first Python file from the terminal

Command to run the file

```
1 python hello.py
2 # or on some systems
3 python3 hello.py
```

- If the terminal is already inside the correct folder, the command should work immediately.
- The output should appear in the terminal below the command.

Expected output

```
> python hello.py  
Hello, Backend World!
```

If students see this output, they have completed the first successful program run.

Second program: reading input from the user

input_example.py

```
1 name = input('Enter your name: ')\n2 print('Welcome,', name)
```

- input() waits for the user to type something.
- The typed value is stored in the variable name.
- print() displays the greeting.

Tiny concept break: what is a variable?

A variable is a named location in memory used to store a value that the program can use later.

Variables in Python

```
1 student_name = 'Mona'  
2 age = 20  
3 print(student_name)
```

Common beginner mistakes

Typical mistakes

- Forgetting quotes around text.
- Using wrong indentation.
- Misspelling python file names.
- Running the command from the wrong folder.

How to react as a student

- Read the error carefully.
- Check line numbers mentioned in the terminal.
- Fix one issue at a time.
- Run the file again after saving.

Mistake example: missing quotes around text

Wrong

```
1 print(Hello)
```

Correct

```
1 print('Hello')
```

- Without quotes, Python thinks Hello is a variable name.
- If no such variable exists, Python raises an error.

Mistake example: running the command in the wrong folder

Error situation

```
1 C:\Users\Student> python hello.py
2 python: can't open file ...
```

Fix

```
1 cd backend_course
2 python hello.py
```

The terminal must be inside the folder that contains the file.

Terminal navigation: the cd command

Useful navigation commands

```
1 cd folder_name
2 cd ..
3 dir    # on Windows
4 ls     # on Linux/macOS
```

- cd changes the current directory.
- cd .. goes one level up.
- dir or ls helps students confirm the files in the current folder.

Class activity 1: students do it now

1. Open VS Code.
2. Open the course folder.
3. Create hello.py.
4. Type the print statement.
5. Save the file.
- 6. Run it in the terminal.**

Class activity 2: personalized greeting

Task code

```
1 name = input('Enter your name: ')\n2 print('Welcome to backend,', name)
```

- Ask each student to replace the text message.
- Check whether the program waits for input.
- Let students run it and show the result to a partner.

Learning Objectives

What students should be able to do after this lesson



- Identify the tools required to start coding in Python.
- Create and run a simple Python file from VS Code or the terminal.
- Use variables, input, conditions, loops, lists, and functions.
- Understand how to debug simple mistakes.

Tools You Need Before Coding

Python interpreter and a code editor

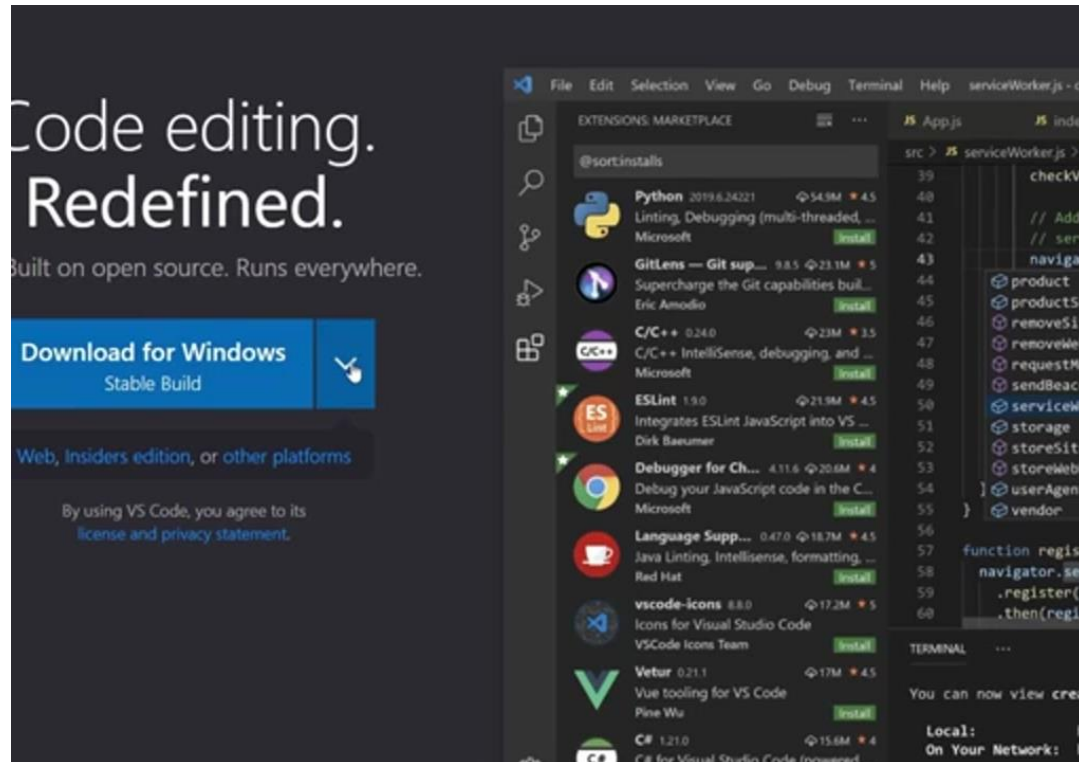


```
1 python --version
```

- Install Python 3 on your computer.
- Use a code editor such as VS Code.
- Keep a simple folder for your course files.
- The terminal will be used to run programs.

Install VS Code

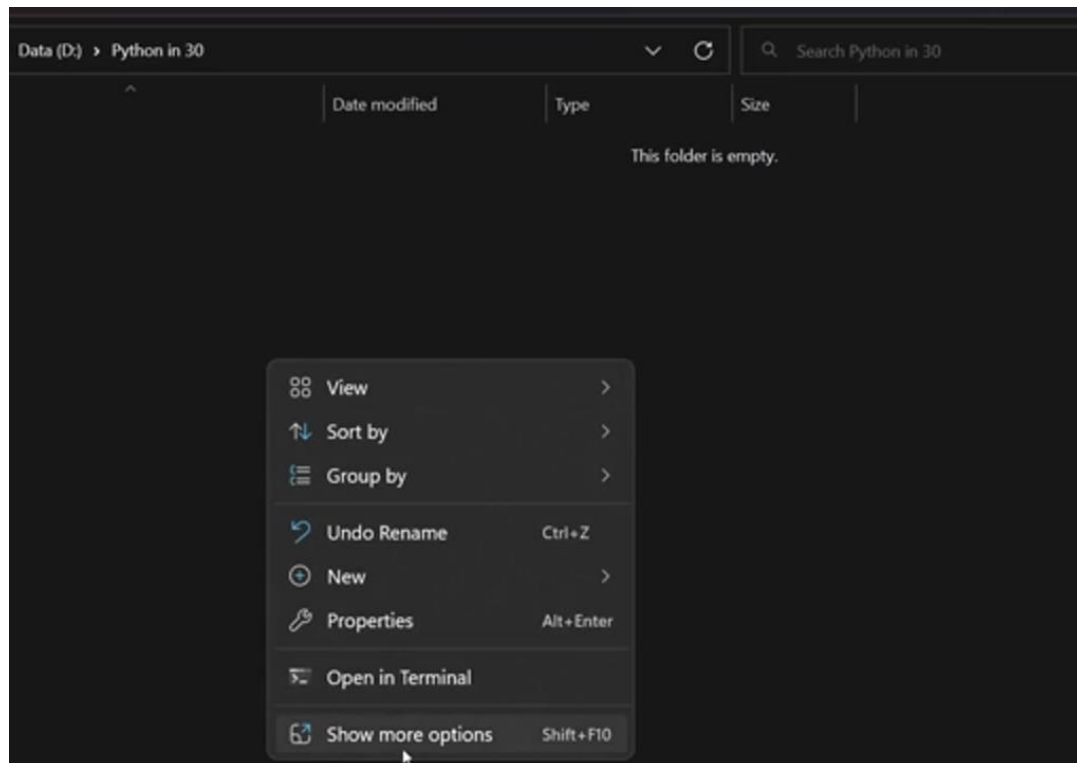
A popular editor for Python beginners



- Download VS Code from the official website.
- Open the installer and complete the setup steps.
- After installation, launch the editor.
- Later, add the Python extension for better support.

Open a Project Folder

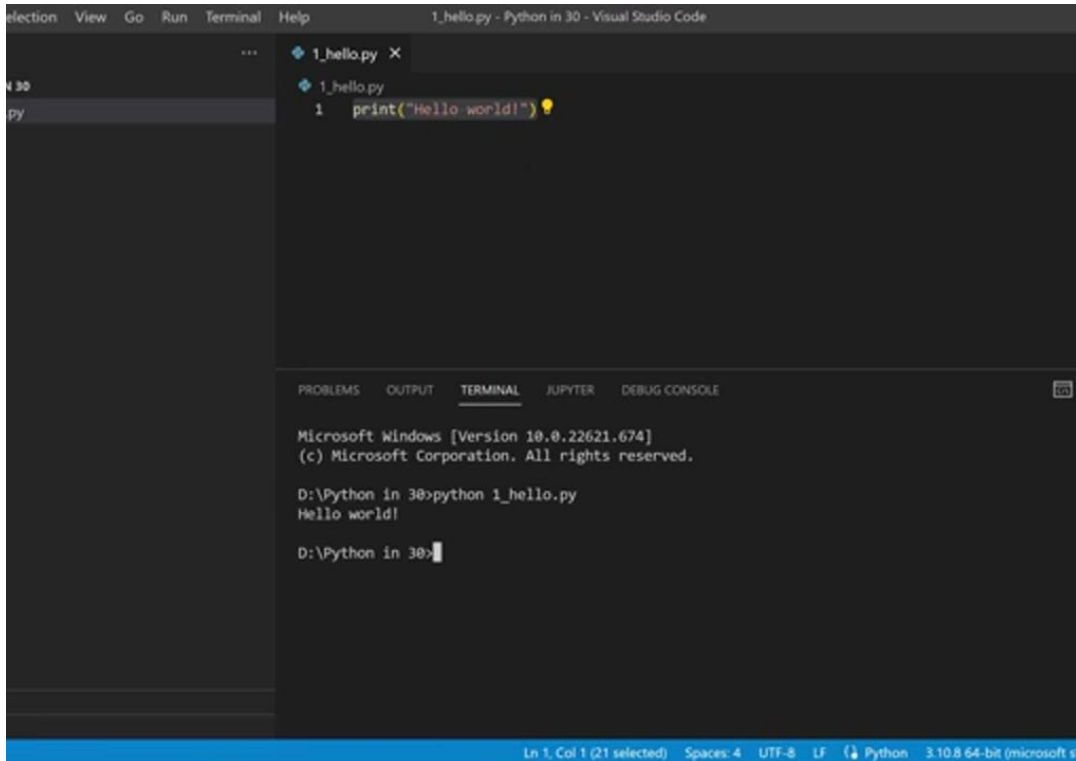
Start with an organized workspace



- Create a clean folder such as `python_basics`.
- Use File → Open Folder in VS Code.
- The Explorer panel will show your files on the left.
- Good organization makes practice easier later.

Create the First Python File

The .py extension tells the editor this is Python code



The screenshot shows the Visual Studio Code interface. The editor window displays a file named `1_hello.py` with the following code:

```
1 print("Hello world!")
```

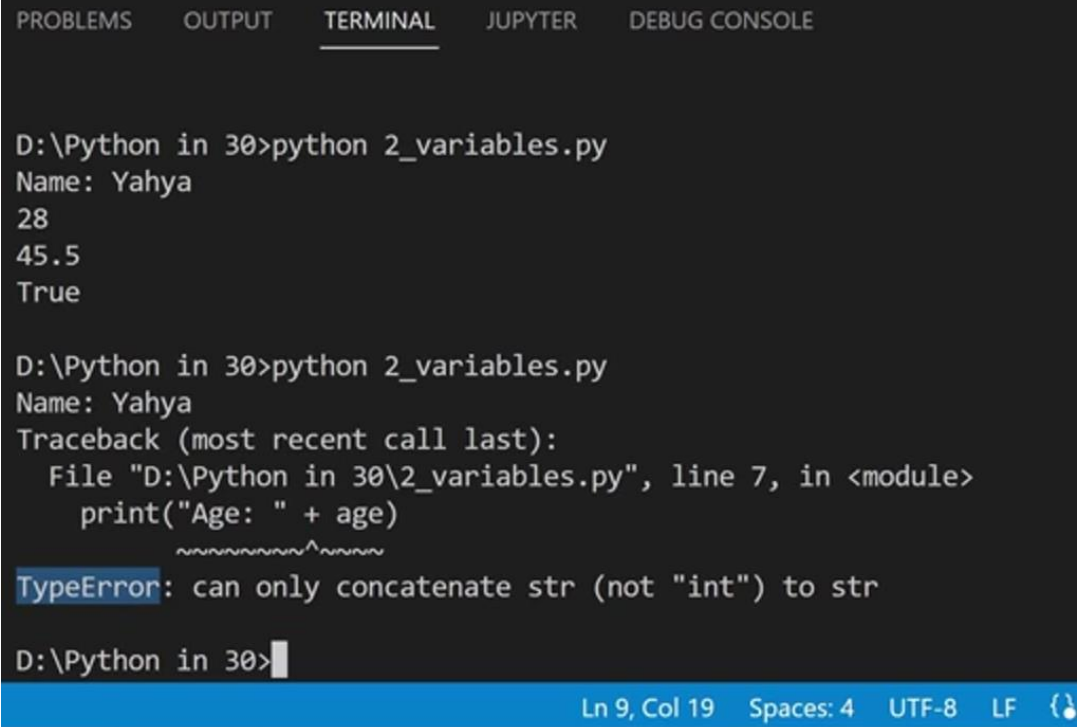
The terminal window at the bottom shows the command `python 1_hello.py` being executed, resulting in the output `Hello world!`.

```
1 print('Hello, Backend World!')
```

- Create a new file and name it `hello.py`.
- Type a simple print statement.
- Save the file before running it.
- Start with very small examples.

Run Python from the Terminal

Executing the file is a core skill



```
PROBLEMS  OUTPUT  TERMINAL  JUPYTER  DEBUG CONSOLE

D:\Python in 30>python 2_variables.py
Name: Yahya
28
45.5
True

D:\Python in 30>python 2_variables.py
Name: Yahya
Traceback (most recent call last):
  File "D:\Python in 30\2_variables.py", line 7, in <module>
    print("Age: " + age)
    ~~~~~^~~~~~
TypeError: can only concatenate str (not "int") to str

D:\Python in 30>
```

```
1 python hello.py
```

- Open the terminal inside VS Code.
- Navigate to the correct folder if needed.
- Use `python hello.py` or `python3 hello.py`.
- The output will appear below the command.

Variables

A variable stores data so the program can use it later

Variables

مكان بنحزّه في الميموري
عشان نشيل فيه بيانات

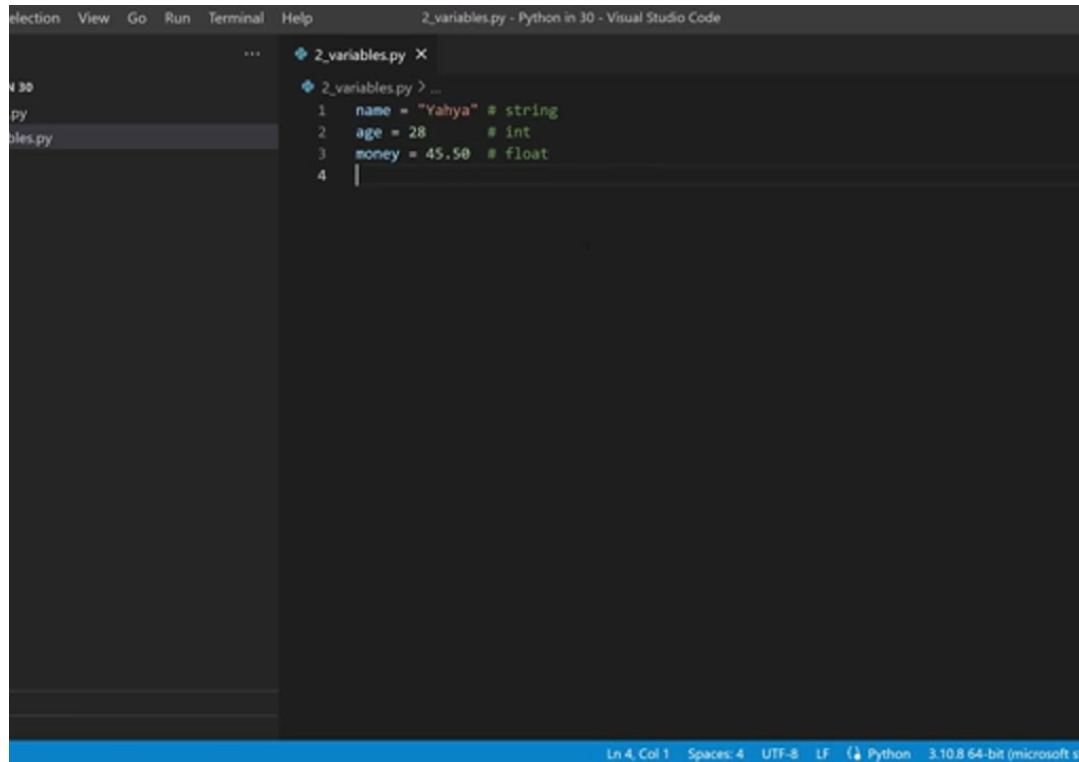


- Variables hold values such as numbers or text.
- Choose meaningful names whenever possible.
- The equal sign assigns a value to a variable.
- Variables make code easier to read and reuse.

```
1 name = 'Yahya'  
2 age = 20
```

Variable Examples in Code

Reading code is as important as writing it



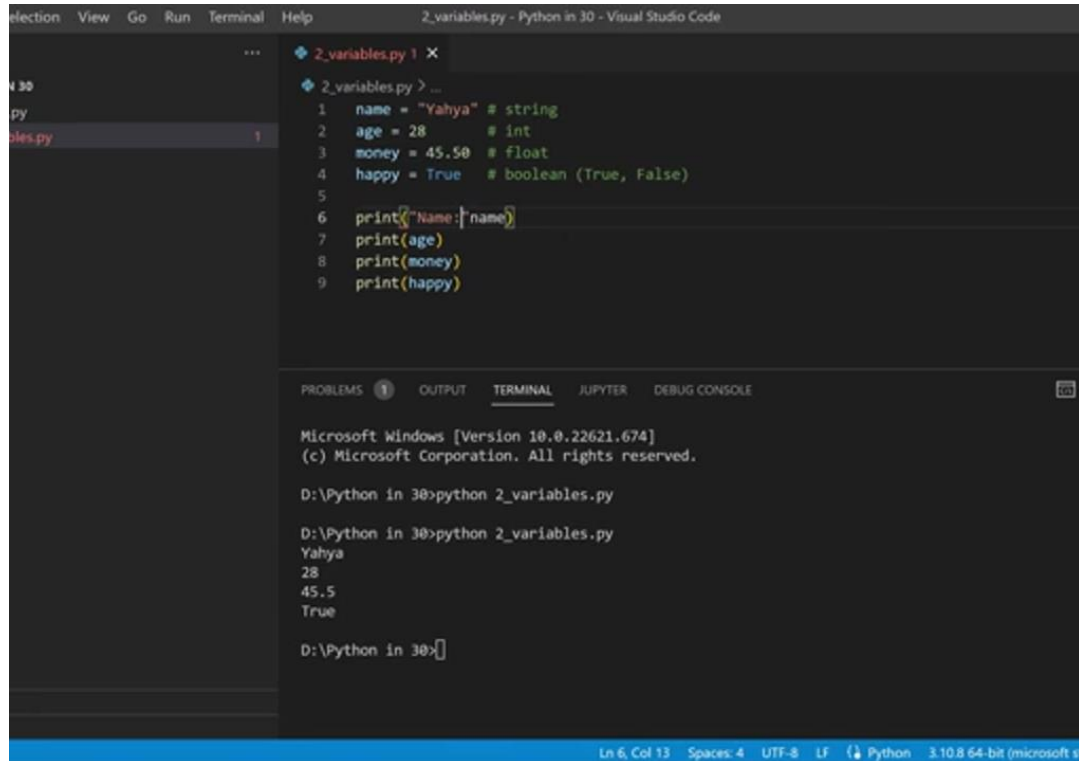
```
1 name = "Yahya" # string
2 age = 28 # int
3 money = 45.50 # float
4 |
```

```
1 name = 'Yahya'
2 age = 20
3 print(name)
4 print(age)
```

- A string is written inside quotes.
- An integer is written without quotes.
- Use `print()` to display variable values.
- You can combine text with variables in output.

Basic Data Types

Python starts with a few essential categories



The screenshot shows a Visual Studio Code editor with a file named `2_variables.py` open. The code defines four variables: `name` (string), `age` (integer), `money` (float), and `happy` (boolean). Each variable is assigned a value and has a type comment. The code then prints each variable. Below the editor, the terminal shows the command `python 2_variables.py` being executed, resulting in the output: `Yahya`, `28`, `45.5`, and `True`.

```
1 name = "Yahya" # string
2 age = 28 # int
3 money = 45.50 # float
4 happy = True # boolean (True, False)
5
6 print("Name:", name)
7 print(age)
8 print(money)
9 print(happy)
```

```
Microsoft Windows [Version 10.0.22621.674]
(c) Microsoft Corporation. All rights reserved.

D:\Python in 30>python 2_variables.py

D:\Python in 30>python 2_variables.py
Yahya
28
45.5
True

D:\Python in 30>
```

```
1 name = 'Ali'
2 level = 2
3 gpa = 3.4
4 is_active = True
```

- str for text values.
- int for whole numbers.
- float for decimal numbers.
- bool for True / False logic.

User Input

Programs become interactive when they ask the user for data

User Input

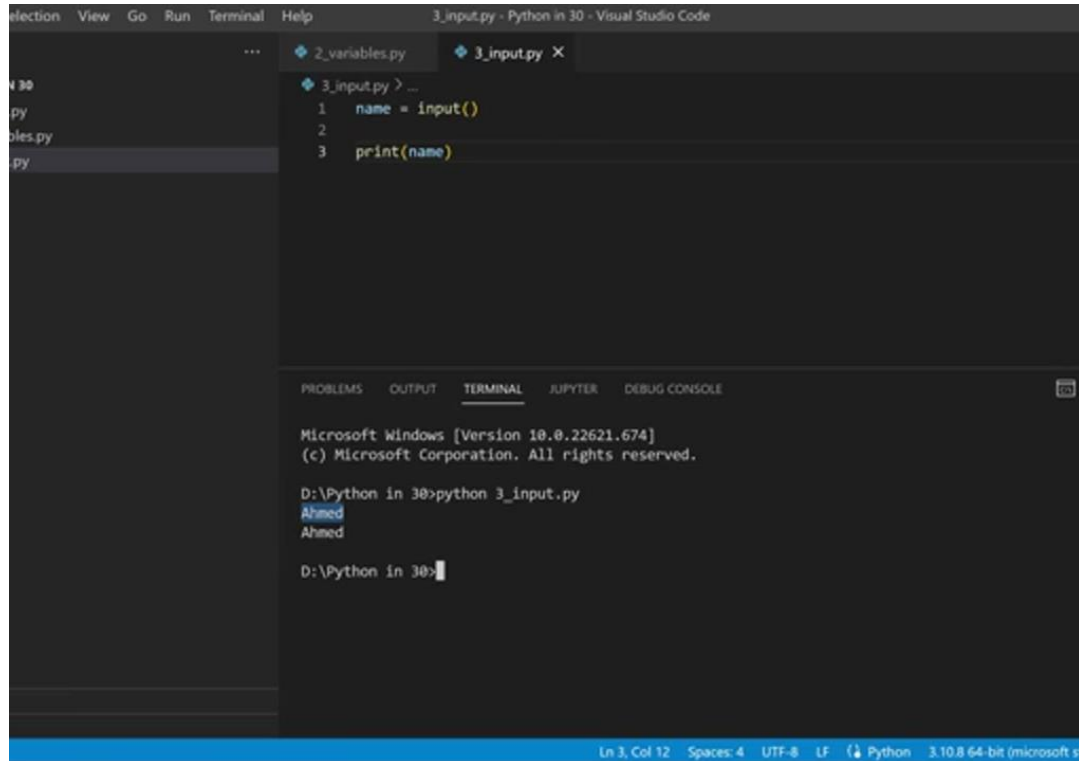
`name = "Yahya"`

```
1 name = input('Enter your name: ')\n2 print('Welcome,', name)
```

- `input()` waits for the user to type something.
- The typed value is stored in a variable.
- The result of `input()` is text by default.
- Use `print()` to show the result.

Using Input in a Real Example

Turn input into a personalized message



The screenshot shows a Visual Studio Code editor with a file named `3_input.py` open. The code in the file is:

```
1 name = input()
2
3 print(name)
```

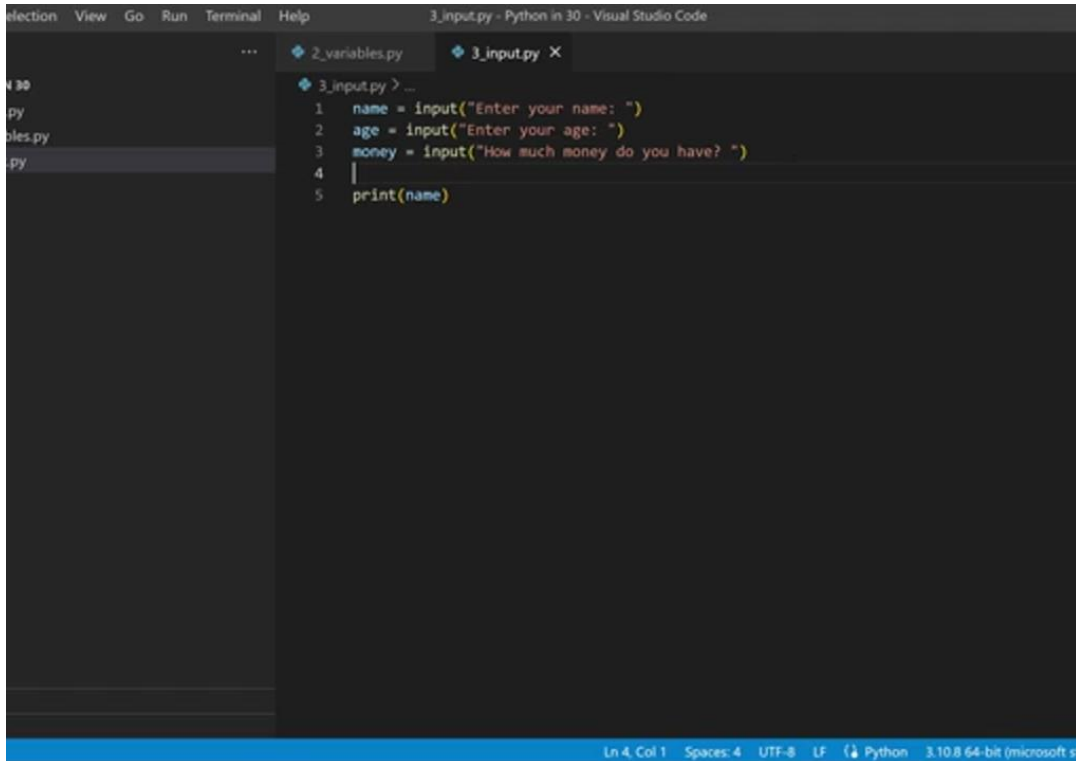
Below the editor, the terminal window shows the command `python 3_input.py` being executed. The output shows the prompt `Ahmed` followed by the printed name `Ahmed`.

- Read a value from the user.
- Store it in a variable.
- Reuse the value in the output.
- This pattern appears in most applications.

```
1 city = input('Enter your city: ')
2 print('Your city is', city)
```

Type Casting

Convert text input into numbers when needed



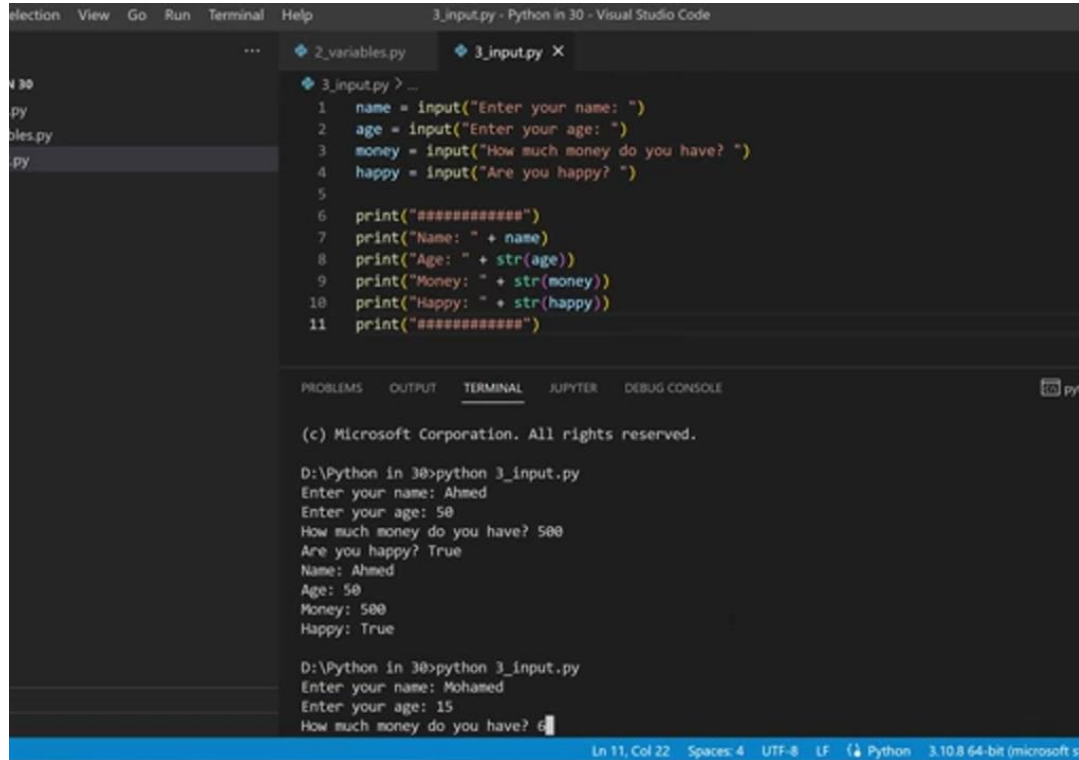
```
1 name = input("Enter your name: ")
2 age = input("Enter your age: ")
3 money = input("How much money do you have? ")
4
5 print(name)
```

```
1 age = int(input('Enter your age: '))
2 print(age + 1)
```

- `input()` returns text, not numbers.
- Use `int()` to convert to an integer.
- Use `float()` to convert to a decimal number.
- Casting is necessary before arithmetic operations.

Comments

Comments explain code to humans and are ignored by Python



```
3_input.py
1  name = input("Enter your name: ")
2  age = input("Enter your age: ")
3  money = input("How much money do you have? ")
4  happy = input("Are you happy? ")
5
6  print("*****")
7  print("Name: " + name)
8  print("Age: " + str(age))
9  print("Money: " + str(money))
10 print("Happy: " + str(happy))
11 print("*****")
```

PROBLEMS OUTPUT TERMINAL JUPYTER DEBUG CONSOLE

(c) Microsoft Corporation. All rights reserved.

D:\Python in 30>python 3_input.py
Enter your name: Ahmed
Enter your age: 50
How much money do you have? 500
Are you happy? True
Name: Ahmed
Age: 50
Money: 500
Happy: True

D:\Python in 30>python 3_input.py
Enter your name: Mohamed
Enter your age: 15
How much money do you have? 6

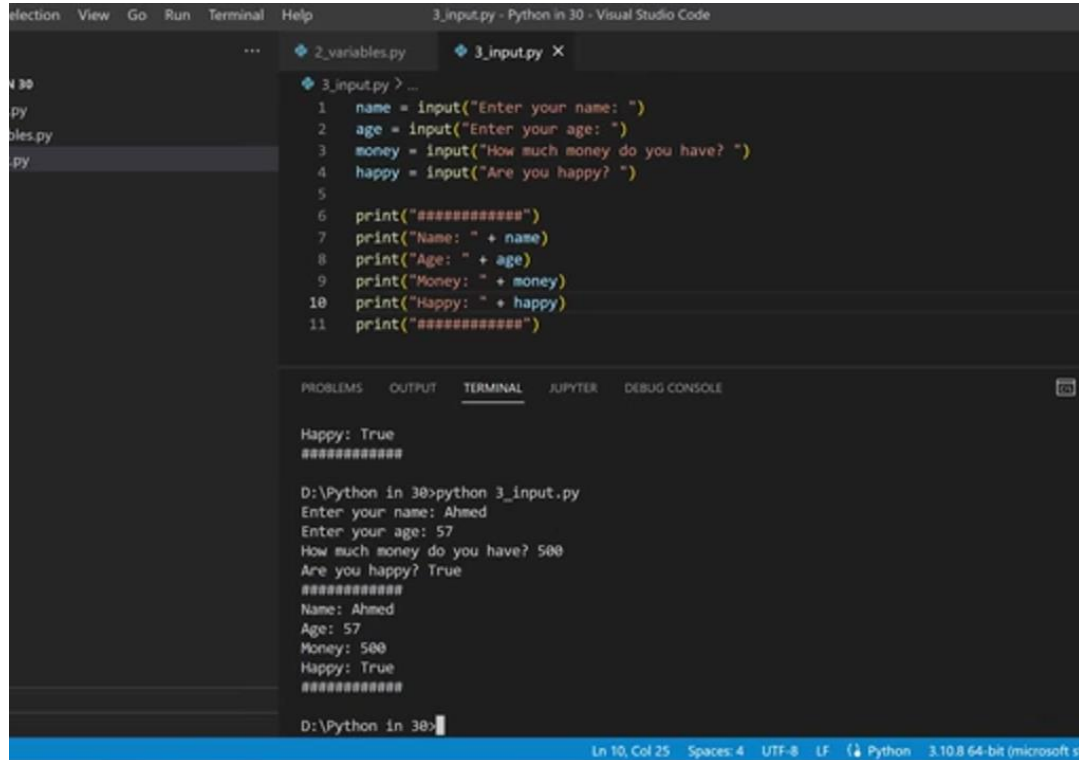
Ln 11, Col 22 Spaces: 4 UTF-8 LF Python 3.10.8 64-bit (microsoft st

```
1 # This program reads the user's name
2 name = input('Enter your name: ')
```

- Single-line comments start with #.
- Use comments to clarify tricky logic.
- Do not write comments that repeat obvious code.
- Good comments help future revision.

Variable Naming Rules

Good naming is both valid and meaningful



```
3_input.py > ...
1  name = input("Enter your name: ")
2  age = input("Enter your age: ")
3  money = input("How much money do you have? ")
4  happy = input("Are you happy? ")
5
6  print("*****")
7  print("Name: " + name)
8  print("Age: " + age)
9  print("Money: " + money)
10 print("Happy: " + happy)
11 print("*****")

PROBLEMS  OUTPUT  TERMINAL  JUPYTER  DEBUG CONSOLE

Happy: True
*****

D:\Python in 30>python 3_input.py
Enter your name: Ahmed
Enter your age: 57
How much money do you have? 500
Are you happy? True
*****
Name: Ahmed
Age: 57
Money: 500
Happy: True
*****

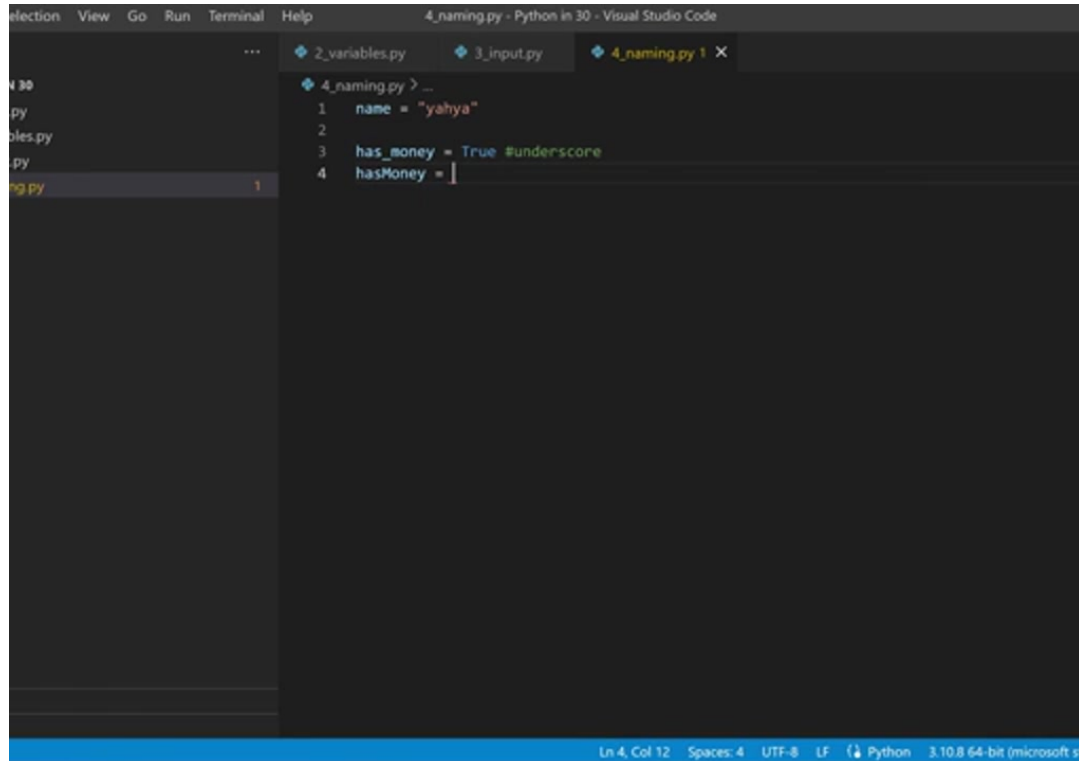
D:\Python in 30>
```

```
1 student_name = 'Mona'
2 score_1 = 90
```

- Names cannot contain spaces.
- Names cannot start with a number.
- Use underscore to separate words.
- Prefer descriptive names over x and y.

Conditions with if

Programs need decision making



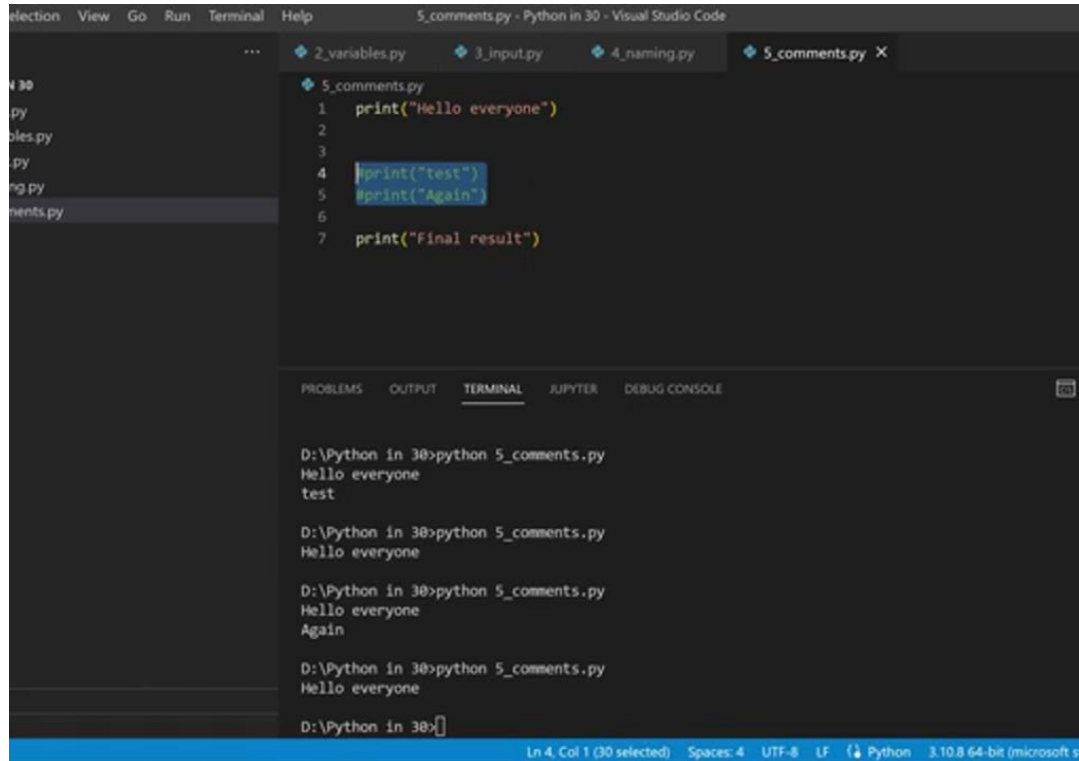
```
1 name = "yahya"
2
3 has_money = True #underscore
4 hasMoney =
```

```
1 age = 20
2 if age >= 18:
3     print('Allowed')
```

- Use if to run code only when a condition is true.
- Conditions often compare values.
- Indentation is required in Python blocks.
- Decision logic is everywhere in real systems.

if / else

Handle both the true case and the false case



```
5_comments.py
1 print("Hello everyone")
2
3
4 print("test")
5 print("Again")
6
7 print("Final result")

D:\Python in 30>python 5_comments.py
Hello everyone
test

D:\Python in 30>python 5_comments.py
Hello everyone

D:\Python in 30>python 5_comments.py
Hello everyone
Again

D:\Python in 30>python 5_comments.py
Hello everyone

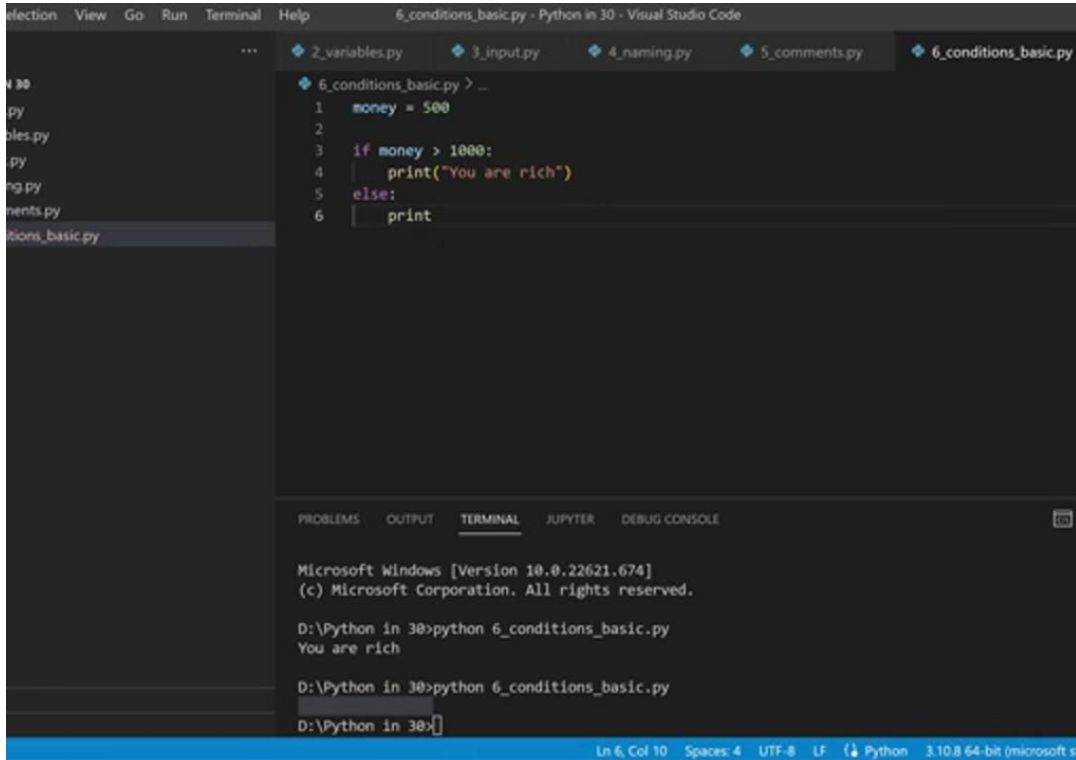
D:\Python in 30>
```

```
1 marks = 45
2 if marks >= 50:
3     print('Pass')
4 else:
5     print('Fail')
```

- if covers the true branch.
- else covers the alternative branch.
- This makes decisions complete and clear.
- Use readable conditions and outputs.

Logical Operators

Combine multiple conditions using and, or, and not



```
File Explorer | Search | View | Go | Run | Terminal | Help
6_conditions_basic.py - Python in 30 - Visual Studio Code

2_variables.py | 3_input.py | 4_naming.py | 5_comments.py | 6_conditions_basic.py

6_conditions_basic.py > ...
1  money = 500
2
3  if money > 1000:
4      print("You are rich")
5  else:
6      print

PROBLEMS | OUTPUT | TERMINAL | JUPYTER | DEBUG CONSOLE

Microsoft Windows [Version 10.0.22621.674]
(c) Microsoft Corporation. All rights reserved.

D:\Python in 30>python 6_conditions_basic.py
You are rich

D:\Python in 30>python 6_conditions_basic.py
D:\Python in 30>

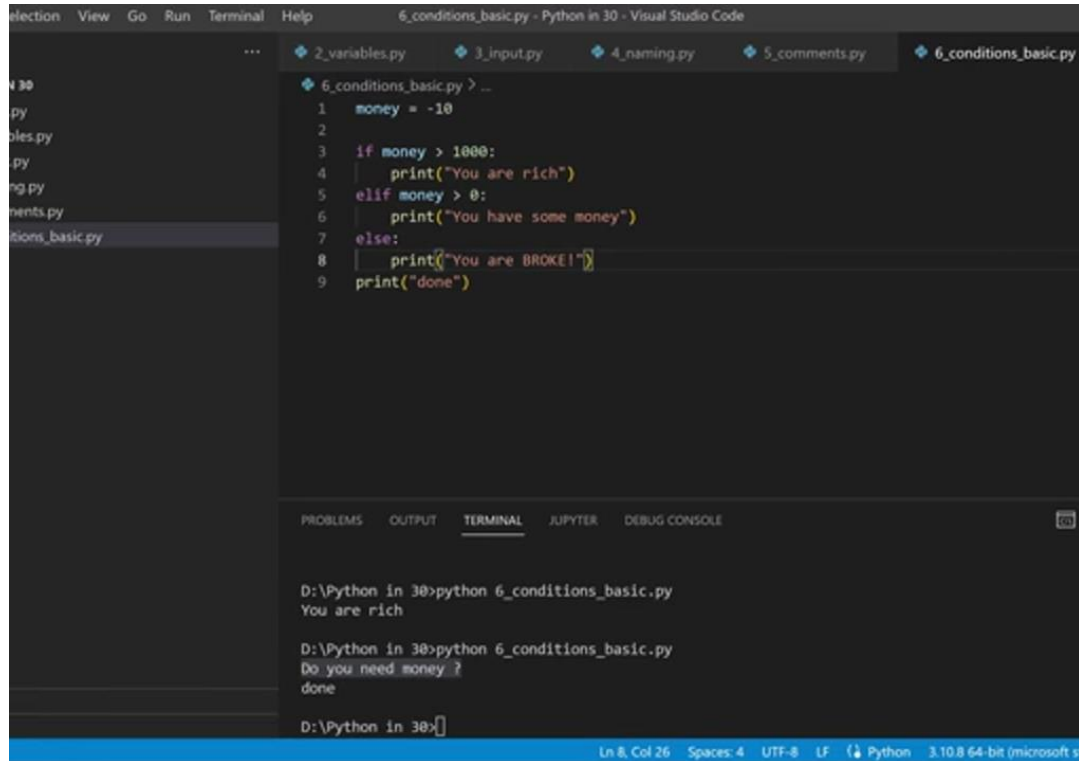
Ln 6, Col 10 | Spaces: 4 | UTF-8 | LF | Python 3.10.8 64-bit (microsoft st
```

```
1 if age >= 18 and has_id:
2     print('Enter')
```

- and means both conditions must be true.
- or means at least one condition is true.
- not reverses a boolean value.
- These operators help build realistic rules.

Loops: for

Repeat an action for a known number of items



The screenshot shows a Visual Studio Code editor with a file named `6_conditions_basic.py` open. The code in the editor is as follows:

```
1 money = -10
2
3 if money > 1000:
4     print("You are rich")
5 elif money > 0:
6     print("You have some money")
7 else:
8     print("You are BROKE!")
9 print("done")
```

Below the editor, the terminal window shows the output of running the script:

```
D:\Python in 30>python 6_conditions_basic.py
You are rich

D:\Python in 30>python 6_conditions_basic.py
Do you need money ?
done

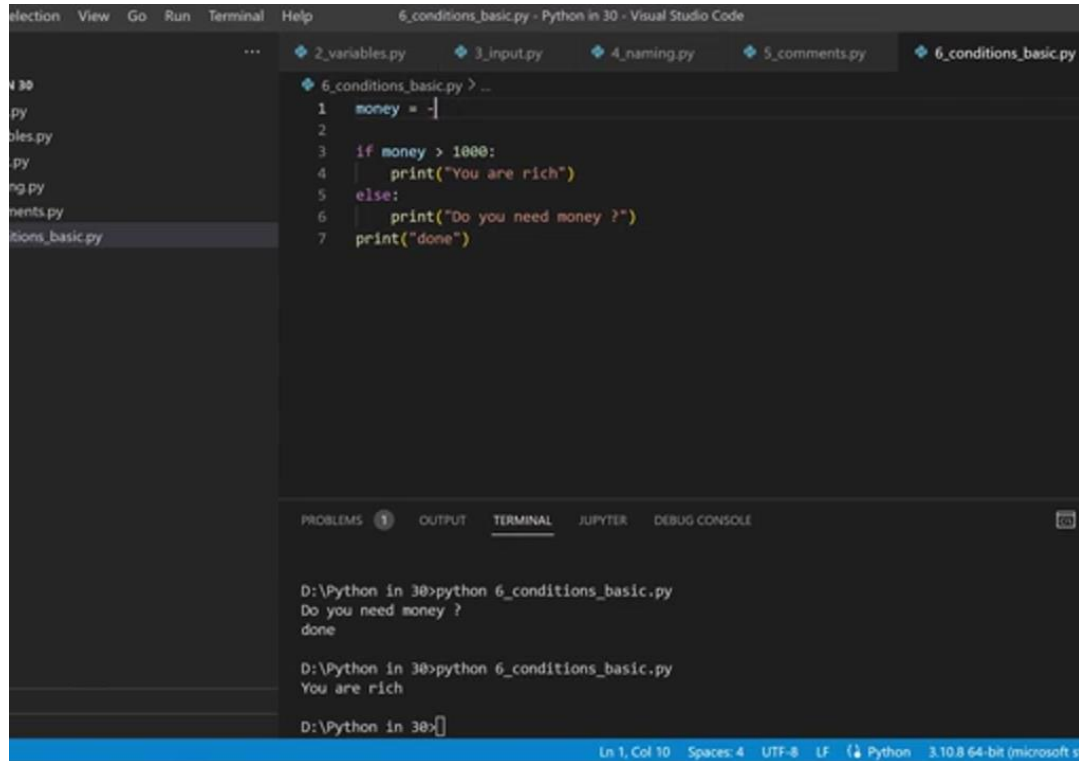
D:\Python in 30>
```

```
1 for i in range(5):
2     print(i)
```

- A for loop is useful with lists and ranges.
- It runs the same block many times.
- The loop variable changes each iteration.
- Use loops to avoid repetitive code.

Loops: while

Repeat while a condition remains true



The screenshot shows a Visual Studio Code editor with a file named `6_conditions_basic.py` open. The code in the editor is as follows:

```
1 money = -1
2
3 if money > 1000:
4     print("You are rich")
5 else:
6     print("Do you need money ?")
7 print("done")
```

The terminal at the bottom shows the output of running the script:

```
D:\Python in 30>python 6_conditions_basic.py
Do you need money ?
done

D:\Python in 30>python 6_conditions_basic.py
You are rich

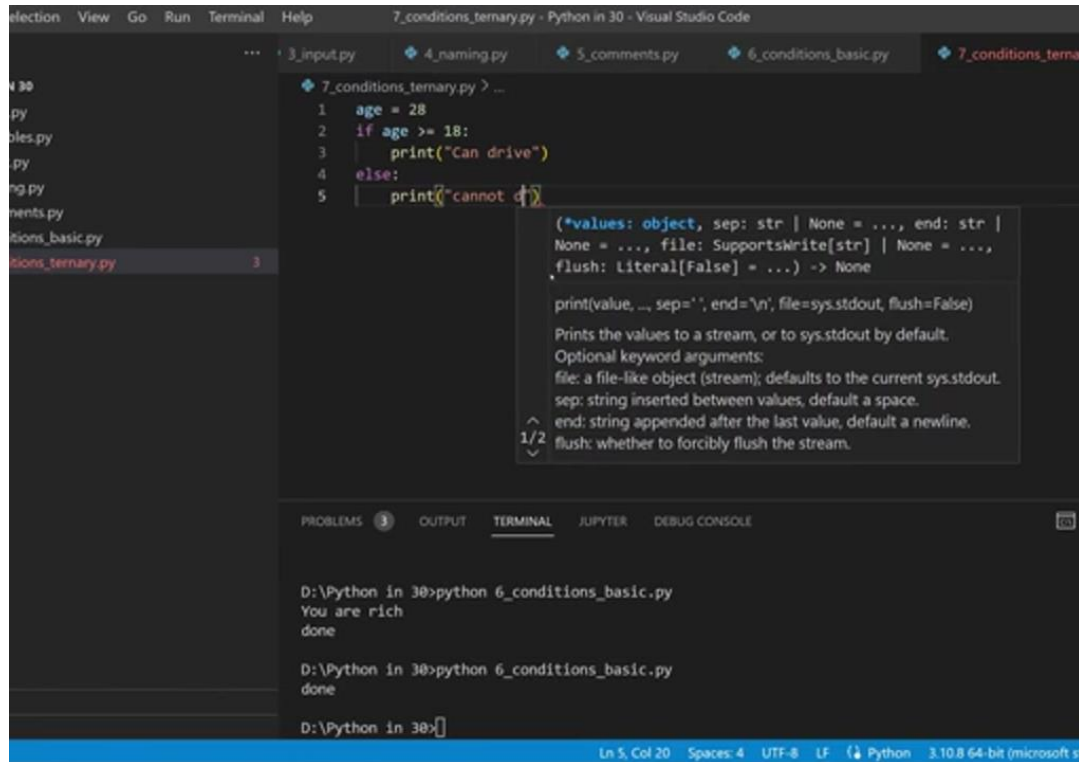
D:\Python in 30>
```

```
1 count = 0
2 while count < 3:
3     print(count)
4     count + 1
```

- while checks a condition before each iteration.
- It is useful when the number of repetitions is unknown.
- The loop must update something inside the body.
- Otherwise, it may become an infinite loop.

Lists

Store multiple related values in one variable



```
1 age = 28
2 if age >= 18:
3     print("Can drive")
4 else:
5     print("cannot drive")
```

(*values: object, sep: str | None = ..., end: str | None = ..., file: SupportsWrite[str] | None = ..., flush: Literal[False] = ...) -> None

print(value, ..., sep=' ', end='\n', file=sys.stdout, flush=False)

Prints the values to a stream, or to sys.stdout by default.

Optional keyword arguments:

file: a file-like object (stream); defaults to the current sys.stdout.

sep: string inserted between values, default a space.

end: string appended after the last value, default a newline.

flush: whether to forcibly flush the stream.

1/2

PROBLEMS 3 OUTPUT TERMINAL JUPYTER DEBUG CONSOLE

D:\Python in 30>python 6_conditions_basic.py

You are rich

done

D:\Python in 30>python 6_conditions_basic.py

done

D:\Python in 30>

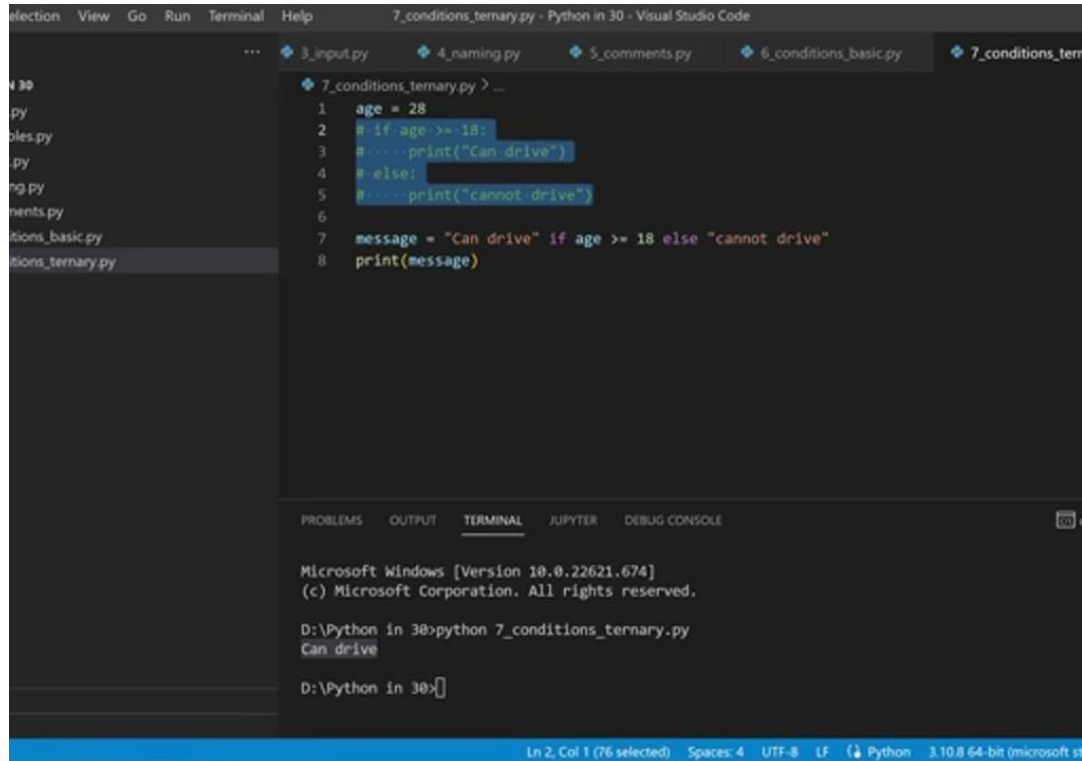
Ln 5, Col 20 Spaces: 4 UTF-8 LF Python 3.10.8 64-bit (microsoft st

```
1 students = ['Ali', 'Mona', 'Sara']
2 print(students[0])
```

- Lists use square brackets [].
- Items are separated by commas.
- Each item has an index starting at 0.
- Lists are one of the most used data structures.

Functions

Group reusable logic into a named block



```
1 age = 28
2 # if age >= 18:
3 #     print("Can drive")
4 # else:
5 #     print("cannot drive")
6
7 message = "Can drive" if age >= 18 else "cannot drive"
8 print(message)
```

Microsoft Windows [Version 10.0.22621.674]
(c) Microsoft Corporation. All rights reserved.

D:\Python in 30>python 7_conditions_ternary.py
Can drive

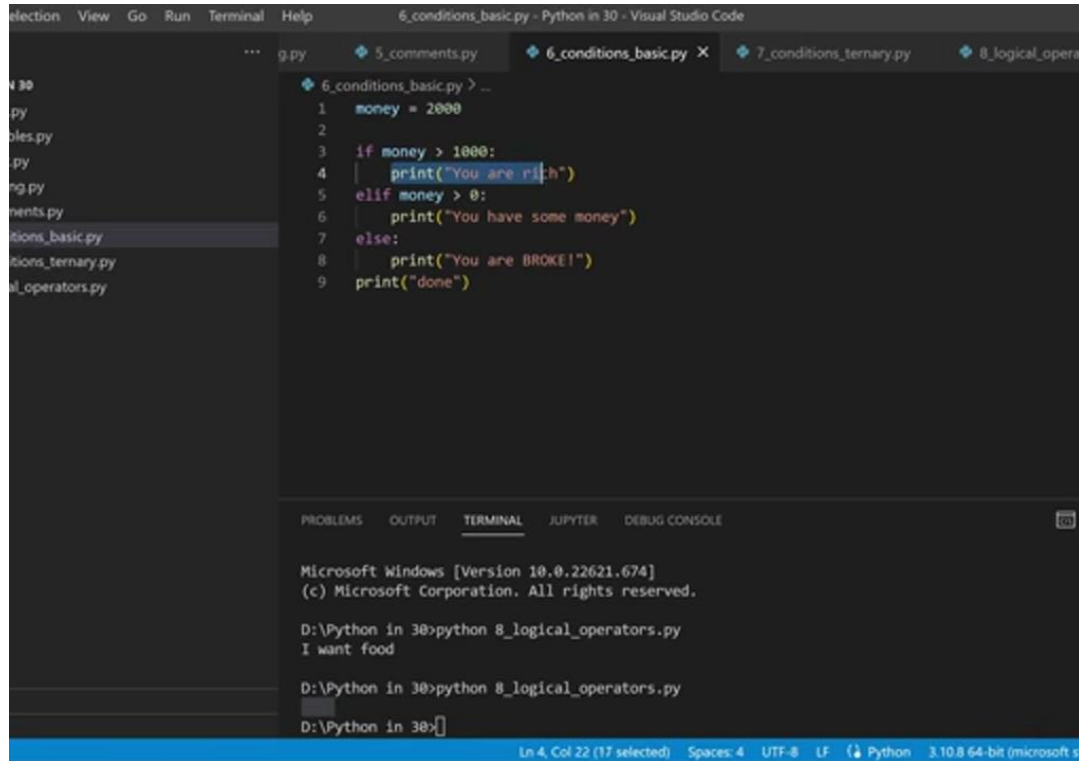
D:\Python in 30>

```
1 def welcome():
2     print('Welcome to Python')
3
4 welcome()
```

- Functions reduce repetition in code.
- Use `def` to define a function.
- A function can receive inputs and produce outputs.
- Functions make programs easier to organize.

Functions with Parameters

Make the same function work with different values



The screenshot shows a Visual Studio Code editor with a file named `6_conditions_basic.py` open. The code in the editor is as follows:

```
1 money = 2000
2
3 if money > 1000:
4     print("You are rich")
5 elif money > 0:
6     print("You have some money")
7 else:
8     print("You are BROKE!")
9 print("done")
```

Below the editor, the TERMINAL panel is visible, showing the output of a command:

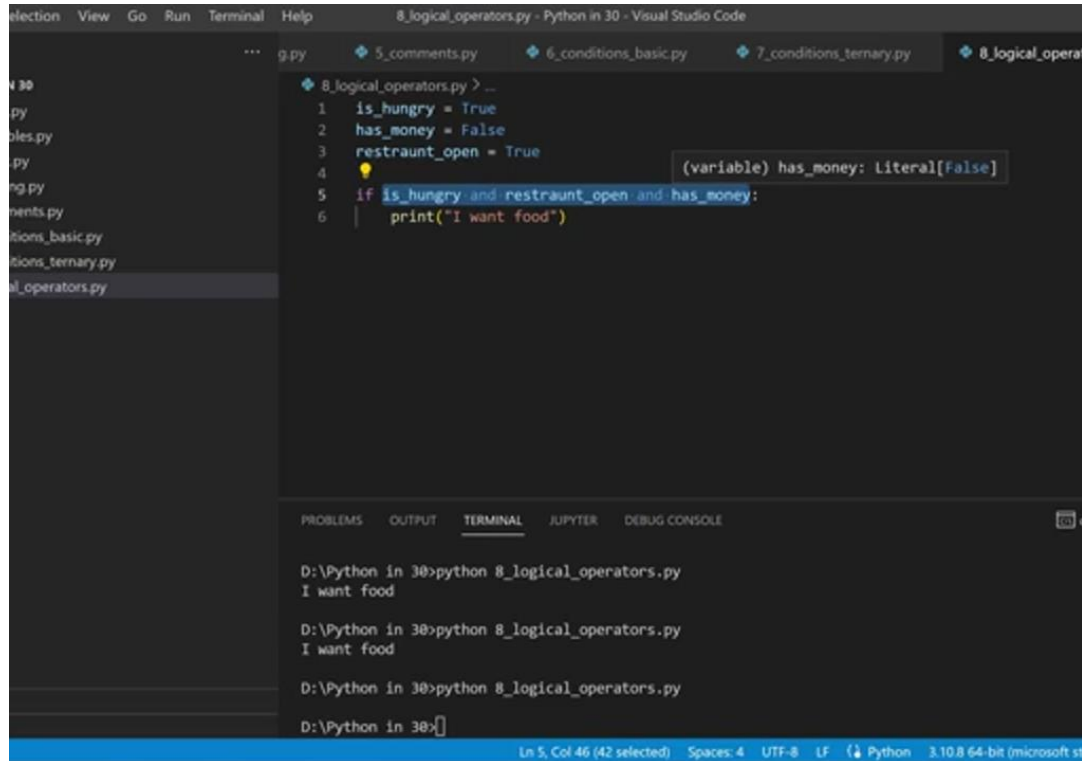
```
D:\Python in 30>python 8_logical_operators.py
I want food
D:\Python in 30>python 8_logical_operators.py
D:\Python in 30>
```

```
1 def welcome(name):
2     print('Welcome', name)
3
4 welcome('Yahya')
```

- Parameters act like inputs inside the function.
- The caller sends values when calling the function.
- This makes one function flexible and reusable.
- Parameters are essential in practical coding.

return vs print

A key distinction for beginners



The screenshot shows a Visual Studio Code editor with a Python file named `8_logical_operators.py`. The code in the editor is as follows:

```
1 is_hungry = True
2 has_money = False
3 restaunt_open = True
4
5 if is_hungry and restaunt_open and has_money:
6     print("I want food")
```

A tooltip is visible over the `has_money` variable, showing its type: `(variable) has_money: Literal[False]`. Below the editor, the terminal window shows the output of running the script:

```
D:\Python in 30>python 8_logical_operators.py
I want food

D:\Python in 30>python 8_logical_operators.py
I want food

D:\Python in 30>python 8_logical_operators.py
I want food

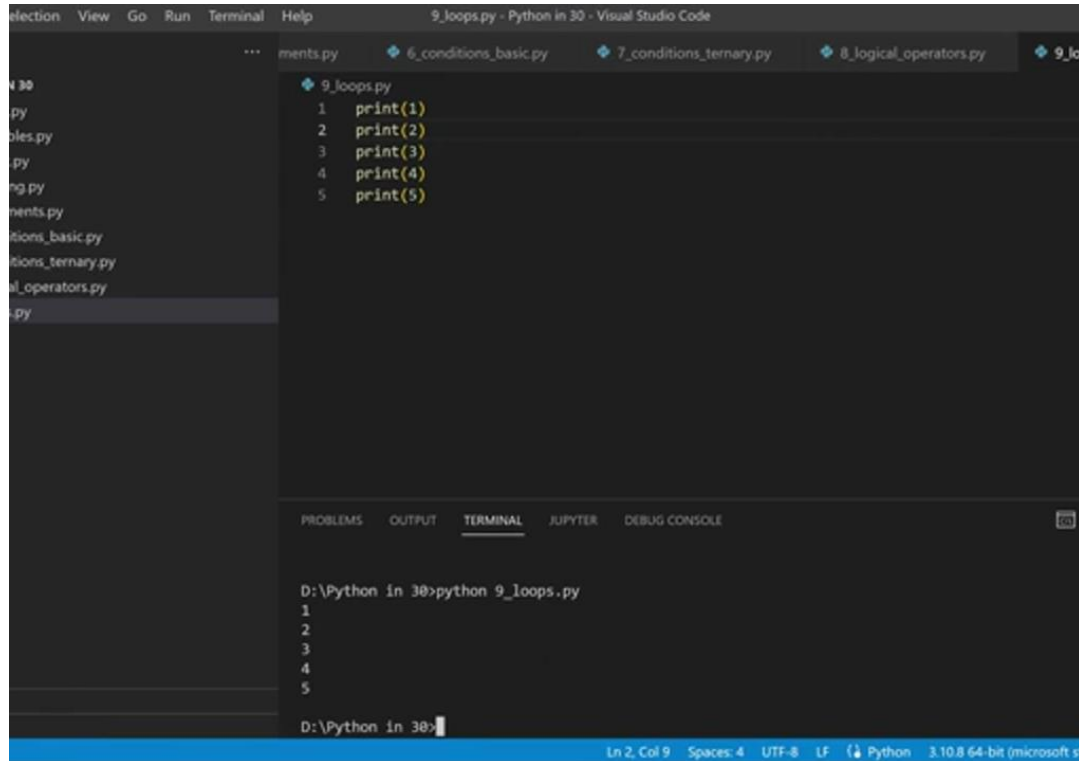
D:\Python in 30>
```

```
1 def add(a, b):
2     return a + b
3
4 result = add(2, 3)
5 print(result)
```

- `print()` displays a value on the screen.
- `return` sends a value back to the caller.
- A returned value can be stored and reused.
- Many bugs come from confusing these two ideas.

Debugging

Read errors calmly and fix one issue at a time



The screenshot shows the Visual Studio Code interface. The editor window displays a file named `9_loops.py` with the following code:

```
1 print(1)
2 print(2)
3 print(3)
4 print(4)
5 print(5)
```

The bottom panel shows the **TERMINAL** tab with the following output:

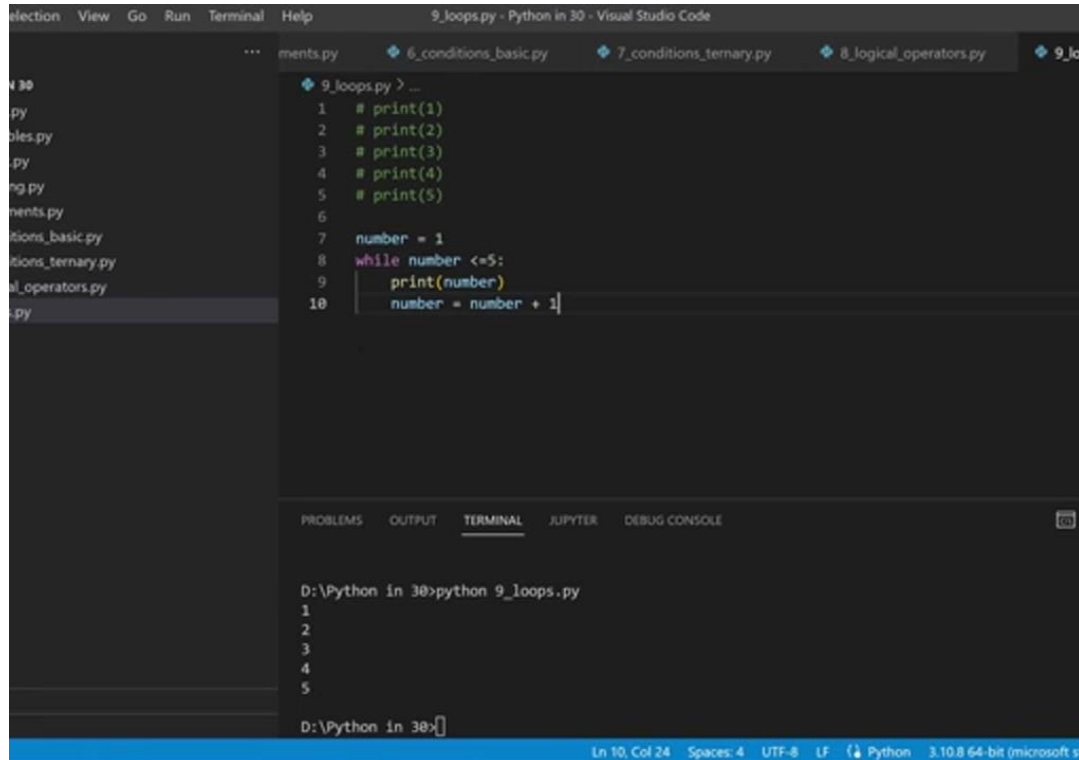
```
D:\Python in 30>python 9_loops.py
1
2
3
4
5
D:\Python in 30>
```

The status bar at the bottom indicates the file is at `Ln 2, Col 9`, using `Spaces: 4`, `UTF-8` encoding, `LF` line endings, and is a `Python 3.10.8 64-bit (microsoft store)` file.

- Syntax errors often come from missing quotes or indentation.
- Tracebacks show the line where Python found a problem.
- Read the error message before changing the code.
- Save the file and run again after each fix.

Mini Practice Task

A small exercise to combine the ideas



```
9_loops.py > ...
1 # print(1)
2 # print(2)
3 # print(3)
4 # print(4)
5 # print(5)
6
7 number = 1
8 while number <=5:
9     print(number)
10    number = number + 1
```

PROBLEMS OUTPUT TERMINAL JUPYTER DEBUG CONSOLE

D:\Python in 30>python 9_loops.py

1
2
3
4
5

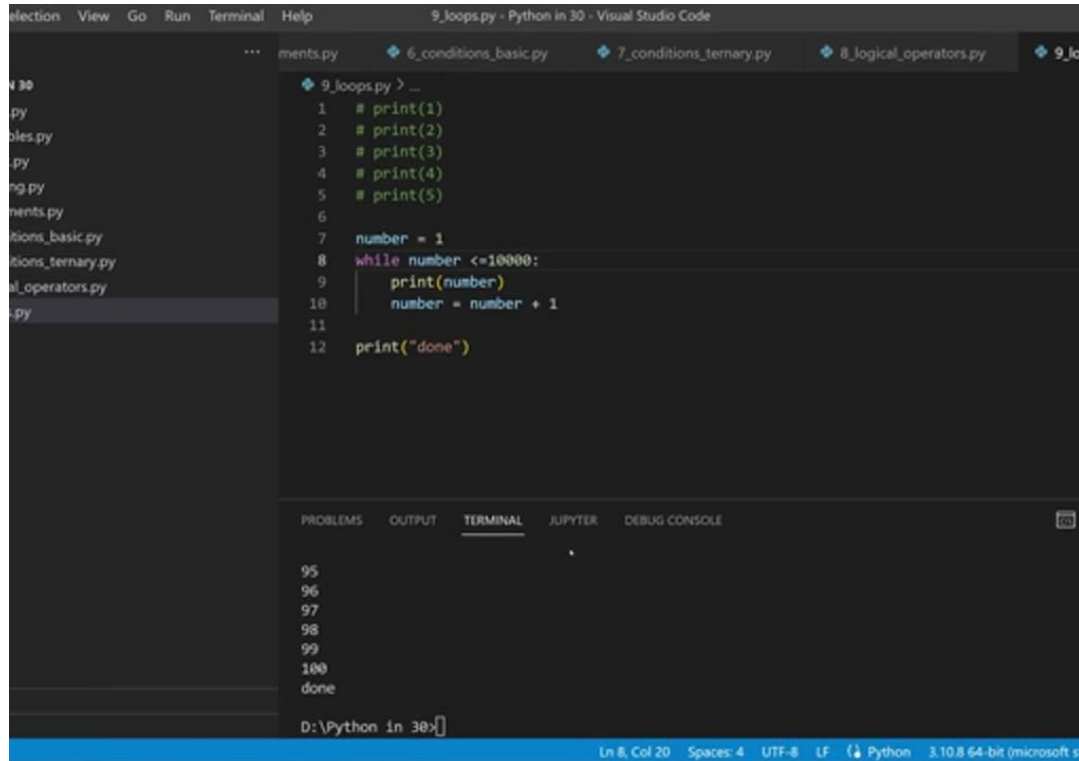
D:\Python in 30>

```
1 name = input('Enter your name: ')
2 age = int(input('Enter your age: '))
3 if age >= 18:
4     print('Welcome', name)
5 else:
6     print('Too young')
```

- Ask the user for name and age.
- Convert age to an integer.
- Print a welcome message.
- Use if to check whether the user is an adult.

Recap and Homework

What to remember after the lesson



The screenshot shows a Visual Studio Code editor window with a Python file named `9_loops.py`. The code contains a `while` loop that prints numbers from 1 to 100. The terminal at the bottom shows the output of the script, displaying the numbers 95, 96, 97, 98, 99, 100, and the word "done".

```
1 # print(1)
2 # print(2)
3 # print(3)
4 # print(4)
5 # print(5)
6
7 number = 1
8 while number <= 10000:
9     print(number)
10    number = number + 1
11
12 print("done")
```

Terminal Output:

```
95
96
97
98
99
100
done
```

- Install the tools and run Python from the terminal.
- Practice variables, input, conditions, loops, lists, and functions.
- Rewrite every example by yourself at least once.
- Try a small homework script using more than one concept.

Think–Pair–Share question

Question: Why do we say that the backend is the “brain” of the system and not the “face” of the system?

Give students 1 minute to think, 1 minute to discuss with a partner, then collect two or three answers.

Mini quiz (oral or written)

1. What is the difference between frontend and backend?
2. What is the purpose of a database?
3. What is the difference between static and dynamic websites?
4. Why is Python a good language for beginners?
5. What command checks the installed Python version?

Lecture recap

- Backend runs on the server side and handles logic and data.
- Web applications involve browser, server, and database.
- Dynamic websites depend on backend code.
- Python is a readable and practical backend language.
- A framework such as Django organizes web development.
- Students should now be able to install Python, install VS Code, and run a simple script.

Homework

1. Install Python and verify the version.
2. Install VS Code and the Python extension.
3. Create a folder named lecture1_practice.
4. Create hello.py and input_example.py.
5. Run both files successfully.
6. Write one paragraph: “What does backend mean?”

Preview of Lecture 2

Next lecture: Python essentials for backend development

We will cover variables, data types, conditions, loops, functions, lists, dictionaries, and simple validation examples.